Robot Chasing

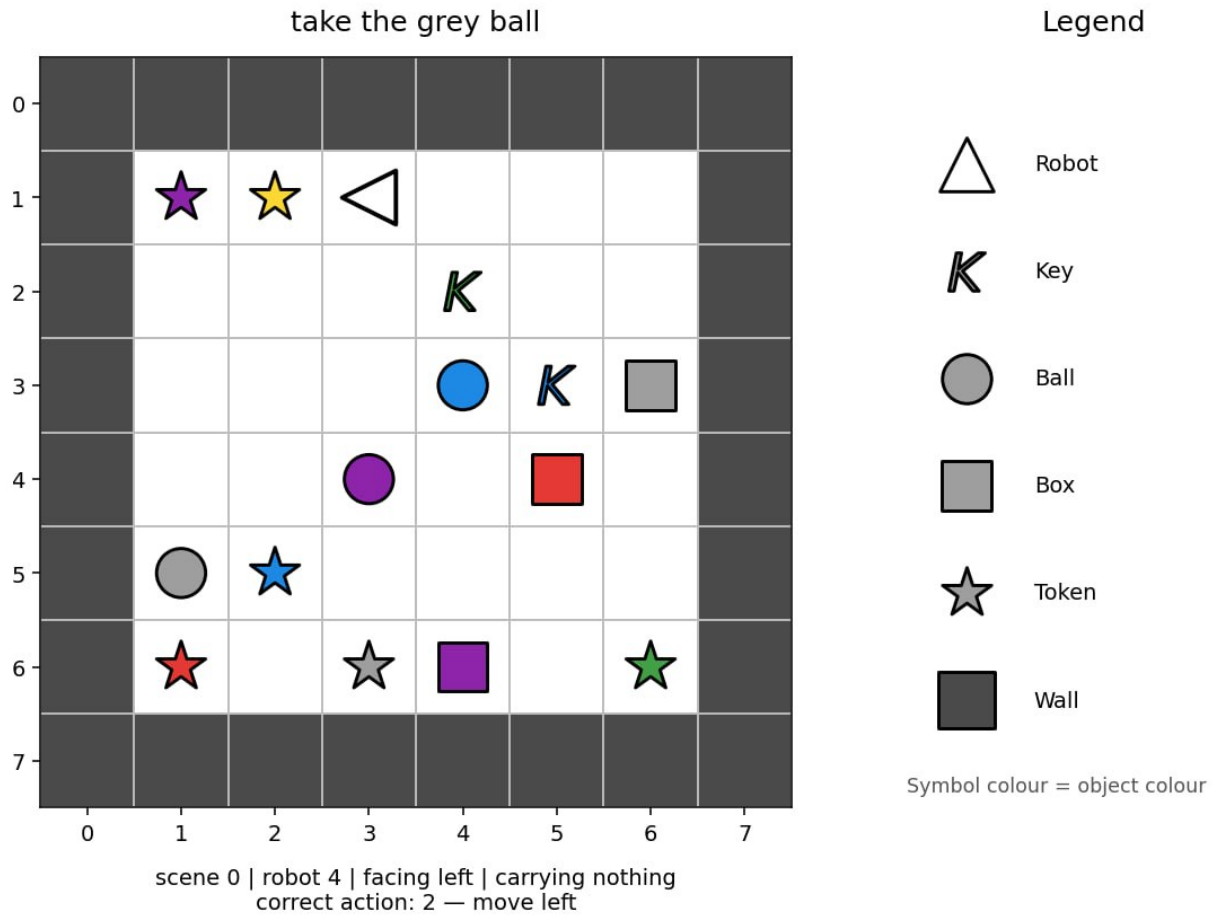
- **Time limit:** 5 minutes
- **Environment:** one GPU (≈16 GB VRAM), no internet
- **Solution size:** `solution.ipynb` ≤ 1 MB
- **Storage:** 5 GB

Task

There are six robots. Each robot operates in a small room represented by a grid. Each room has a `6×6` playable area surrounded by walls, so the full `image` array has size `8×8` (playable area + walls).

Each robot receives an English instruction describing a task. The snapshot may be taken at any point while the robot is carrying it out. Your goal is to predict the robot's next action.

Robots do not always follow the shortest path. Robot 0 may behave differently from Robot 1, but each robot follows its own consistent pattern. Use the training examples, which include the correct next actions, to learn these patterns.



There are three types of missions:

- **go to** an object, for example "`approach the red ball`";
- **pick up** an object, for example "`grab the blue key`";
- **put one object next to another**, for example "`place the red box beside the green ball`".

The same instruction can be written in several ways. The test set may contain new combinations of familiar phrases, colours and object types. However, every word, phrase pattern, colour, object type and mission type used in the test set also appears in the training set.

Each sample has the following fields:

Field	Meaning
`robot_id`	which of the 6 robots this is (`0`–`5`)
`image`	the room, an `8×8×2` integer array where channel 0 holds categorical object_idx (e.g., 1=empty, 2=wall, 10=robot) and channel 1 holds categorical colour_idx (0–5).
`direction`	the direction the robot currently faces
`mission`	the visible natural-language instruction

Field	Meaning
`carrying`	`null` or `[object_idx, colour_idx]` for the carried object

Rows are independent snapshots in random order. They do not form episodes, and no previous observation or action is available at evaluation time.

The provided `visualize_dataset.ipynb` lets you inspect the observations available to the model in different situations.

Grid encoding

`image[row][column] = [object_idx, colour_idx]`. The first index is the row from top to bottom, and the second is the column from left to right. The array includes the outer wall border, so the navigable interior is `6×6`.

Object ids:

id	object
1	empty cell
2	wall
5	key
6	ball
7	box
10	robot
11	token

Tokens may appear in the room but are never named in missions.

Colour ids are `0` red, `1` green, `2` blue, `3` purple, `4` yellow and `5` grey. The colour channel has no meaning for empty cells and walls.

The image has only the two channels above. The robot's direction is provided once, in the top-level `direction` field; it is not duplicated inside `image`.

Actions

For codes `0–3`, movement actions use the following absolute mapping:

action	meaning
0	move up
1	move down
2	move left
3	move right
4	pick up
5	drop

The `direction` field indicates current facing orientation using: 0 = Up (row - 1), 1 = Down (row + 1), 2 = Left (col - 1), 3 = Right (col + 1).

A movement action first turns the robot to that absolute direction and then attempts to move it by one cell. A wall or object may block the move, but the direction still changes. `pick up` and `drop` act exclusively on the adjacent target cell defined by direction (e.g., if direction=0, it acts on (row - 1, col)).

Dataset

You receive two folders:

Folder	Rows	`labels.json`?	Use it to
`dataset/train/`	60,000	included	train your model
`dataset/test_public/`	3,600	included in the development copy	run and self-score your pipeline

Each folder contains `observations.json`, a JSON list of the samples described above. `labels.json` is an aligned JSON list of actions (`0`–`5`).

The training set contains exactly 10,000 rows per robot and 20,000 rows from each task family. The public test contains 600 rows per robot. Wrap `image` with `numpy.asarray(...)` if you need an array.

At grade time, `dataset/test_public/` is transparently replaced by a hidden set of 3,600 observations in the same format, but without `labels.json`. The public leaderboard uses `test_leaderboard_a`; the final ranking uses `test_leaderboard_b`. A notebook that unconditionally reads test labels will fail. Read labels only from `dataset/train/`.

Output

Write `predictions.json` in the notebook's working directory. It must be a JSON list containing one integer action (`0`–`5`) per row of `dataset/test_public/observations.json`, in the same order. For a hypothetical test set containing six samples, a valid output would be:

```
[0, 3, 2, 2, 5, 4]
```

A missing or invalid JSON file, a wrong number of predictions, a non-integer value, or an action outside `{0,1,2,3,4,5}` is rejected without a score.

Scoring

The scoring is **mean per-robot accuracy** on a `0–100` scale. Accuracy is first computed independently for each robot, then averaged over all six robots. Every robot therefore has equal weight.

How to submit

1. Open `solution.ipynb` and run all cells.
2. Confirm that it writes `predictions.json` with 3,600 predictions for the public test set.
3. Improve the model if you want; the provided baseline only demonstrates the required input and output format.
4. In the JupyterLab Git tab, stage and commit `solution.ipynb`, then push it.
5. Return to the Contest page and click **Submit**.

Submit exactly one file named `solution.ipynb`.